FemboyEngine Standard

Version: 2.0 (naming and module standard)

This document is the canonical source for naming, module boundaries and game architecture. Source-controlled documentation is Markdown because it is diffable, reviewable and has no binary merge conflicts. DOCX/HTML are generated distribution artifacts, not source files.

1. Product and namespaces

Item	Rule	Example
Product	FemboyEngine	FemboyEngine/Engine.hpp
Runtime namespace	Fem	Fem::Engine, Fem::Input
Math library	FemMath	FemMath::Vec2, FemMath::Pi
Game namespace	Project name in PascalCase	Neon

The transitional alias namespace Femboy = Fem; exists only for old external code. New code must not use it.

2. Files and directories

- Public engine headers live under engine/include/FemboyEngine/.
- FemMath lives under engine/include/FemMath/.
- One primary type per file; the file name matches the type: DrawList.hpp, PlayerController.hpp.
- Use PascalCase for file names.
- Keep acronyms uppercase in file and type names: DPI.hpp, ECS.hpp, UI.hpp.
- Do not create vague files such as Common.hpp, Misc.hpp or Utils.hpp in new code. Split them by domain: Math.hpp, Geometry.hpp, Text.hpp.
- Do not commit build trees, CMake distributions, generated assets, runtime saves, crash logs or editor state.

3. Identifiers

- Types, structs, classes, enums: PascalCase. Example: PlayerState, Screen.
- Functions and methods: PascalCase. Example: Update, MouseWheel.
- Variables, members and parameters: PascalCase. Example: CurrentScreen, DeltaTime, FrameDrawList.

- Boolean names use Is, Has, Can or Should: IsVisible,

HasFocus, ShouldReduceMotion.

- Enum values: PascalCase: Screen::Playing.
- Constants: scoped constexpr in PascalCase: Limits::MaxEntities,

FemMath::Pi.

- Macros: FEM_ plus uppercase: FEM_ASSERT, FEM_PLATFORM_WINDOWS.
- New code must not use trailing underscores. A trailing _ is legacy from the

old private-field convention and is removed as code is touched.

- Do not name a variable after its type. Use a role suffix:

Hud Interface, not Hud Hud; CurrentScreen, not Screen Screen.

- Interfaces may keep the established I prefix (IRenderer) until a planned

interface rename. New interfaces should prefer a noun plus Interface.

4. Modules

Start with four clear engine modules:

Module	Responsibility
FemCore	Result, logging, time, config, version, base objects
FemMath	vectors, rectangles, colours, angles, geometry, random
FemRender	DrawList, camera, projection, renderer backend
FemWorld	World, ECS, scene, prefab, physics, sockets

Additional modules may be split later: FemAssets, FemInput, FemAudio,

FemUI, FemScript, FemNet.

FemMath must not depend on ECS, scene or renderer headers. It may temporarily

use SFML value types through aliases:

```
namespace FemMath
{
    using Vec2 = sf::Vector2f;
    using Color = sf::Color;
    struct Rect : sf::FloatRect { /* {x,y,w,h} helpers */ };
}
```

5. Game architecture

Game is a composition root, not the world, player and HUD at once.

Layer	Owns
Fem::Engine	window, renderer, input, audio, assets, engine services
Fem::World	entity lifecycle, views, spawn/destroy/clear
GameMode	wave pacing, enemy selection, spawn points, level rules
PlayerState	player-owned mutable state: input, profile, skills, position, cooldowns
PlayerController	input-to-intent mapping, trigger latching, weapon and ammo actions
Hud	menus, HUD widgets, cursor state and presentation
Game	wiring, app flow, level transitions, composition

Rules:

- Game code must not call registry.create/destroy/clear/emplace directly.

Use World.

- Game code must not call Engine::GetRegistry() for gameplay lifecycle.

World::GetRegistry() exists only for legacy systems.

- Player state must live in PlayerState, not in Game.
- HUD must read game state; it must not own gameplay simulation.
- GameMode issues spawns through Game/World; it must not contain rendering.

6. Windows executables

Games and the editor are GUI applications; tests are console applications.

```
if(WIN32 AND TARGET SFML::Main)
    set_target_properties(MyGame PROPERTIES WIN32_EXECUTABLE TRUE)
    target_link_libraries(MyGame PRIVATE SFML::Main)
endif()
```

SFML::Main supplies WinMain and forwards to the normal main(). Without the

Windows GUI subsystem the executable gets a console window when launched from

Explorer. Test executables stay console (Windows CUI) so their output remains

visible.

7. Documentation

- docs/STANDARD.md and docs/STANDARD.ru.md are the canonical standard.
- Topic documents such as RENDERING.md and CLASS_DEFAULTS.md are reference

material, not a second standard.

- Generated DOCX/HTML/PDF belongs in docx/ or another artifact directory and

must not be committed as source.

- Every public engine API must have a short purpose statement and a usage

example when the behaviour is not obvious.

- Russian and English standard documents must stay structurally equivalent.

7. Memory and tests

Every new engine allocation-heavy subsystem should provide:

- a deterministic unit test for construction, destruction and clear;
- a stress test for repeated create/destroy cycles;
- an optional process-memory snapshot for manual diagnostics;
- no leaks in sanitizer/debug runs once the memory tooling lands.

The memory-tracking tool is not part of the current standard surface yet; it is

the next engine infrastructure task.

8. Current migration state

- Include root: FemboyEngine/.
- Runtime namespace: Fem.
- DrawList renamed to DrawList; compatibility alias remains.
- FemMath exists.
- World exists.
- PlayerState, LevelState, PlayerController and GameMode exist.
- Ui renamed to Hud.
- Legacy trailing-underscore fields remain in Game; remove them as those

areas are touched.